Serial Protocol Decode — DMM6500

A UART decoder that runs **on** a Keithley DMM6500 bench multimeter. It digitizes the line with the instrument's own digitizer, recovers the baud rate, frame format and idle polarity from the signal, and shows the bytes on the front panel as text or hex. No host, no logic analyser, one probe.

Ian Ameline · **version 0.9 — beta** · MIT licence (see LICENSE) · user manual

Beta status. Everything the manual claims was measured on the bench, and the release gate below is what measures it — it must pass with zero failures before a build goes out. One rough edge worth naming here: **flow control** is verified electrically — 4.92 V, ~10 µs, one credit pulse per armed capture — but has never been closed as a loop against a device that actually waits for it, so the "unlimited with flow control" claim is sound by construction rather than demonstrated. The pulse width is **not settable** on this firmware — `trigger.extout.pulsewidth` is nil on 1.7.17a — and the default measures **9.4–9.8 µs**, so a device waiting for a credit needs an edge-triggered input rather than a polling loop.

The front-panel TRIGGER key is the stop control, and it is confirmed on hardware. A touch press cannot be delivered while a script runs, so the app cannot see a Cancel button; the TRIGGER key is latched by the trigger hardware instead, which keeps working while the interpreter is busy. Measured with `tools/bench_cancelkey.py`: a press during a deliberate 10 s Lua spin was still seen by the poll after it, at a cost of 1.06 ms per poll and zero event-log entries. That is what makes a one-press recording honest rather than a hang. The same key also *gates* an armed capture when `Options ▸ Trigger = Trigger key`, which took its own purpose-built test (`tools/bench_trigkey.py --quiet-line`) because timing-based tests could not tell a prompt press from a free-running capture.

How much it captures, and how many presses. A screenful is ~240 bytes; a recording holds **8 192 or 32 768 bytes** to a file, and **one press** records it, decodes all of it and files it — no stepping, where a full buffer once cost twelve presses. Beyond one window, flow control makes the window a chunk size rather than a total: one press then credits the device, records, decodes and credits again until it stops sending, so an unlimited amount can be captured losslessly **with no interaction per window** by a device built to wait for the credit — bounded at 32 windows or 20 minutes per press, then Mode and Capture resume the same conversation in the next file – the sender is still waiting for its credit, so nothing is lost in the gap. There is deliberately no uncapped mode: continuous decode-to-file is drain-bound at 2400 baud, too slow to be worth a mode. The arithmetic is in docs/REFERENCE.md.

Ships as `Serial_Decode.tspa`, installed from a USB key through the instrument's own **Manage Apps** screen. Written in TSP — Lua **5.0.2** embedded in the instrument firmware, which is why the sources avoid `#`, `%`, `string.gmatch` and bitwise operators throughout.

Before releasing it to anyone

```
python3 tools/release_sweep.py
```

That is the gate. It runs every check in one go and writes an auditable record to `out/release/<timestamp>/` — a `REPORT.md`, a `summary.json`, the full output of every stage, every front-panel

screenshot taken, and the exact `.tspa` and `MANUAL.pdf` the results describe with their SHA-256s. It exits non-zero if any gate fails.

It needs a freshly power-cycled DMM6500. `sdec.start()` builds the display objects and the firmware does not fully return the pool, so the app refuses a second build in one power cycle. The sweep checks this up front and refuses with the remedy rather than failing forty minutes in. It also needs the SDG2122X generator with the stimulus waveforms loaded (`bench_matrix.py --upload` puts them there) and only one client may hold the DMM's control socket at a time.

For a code change, the offline half runs in under ten seconds and needs no instruments:

```
python3 tools/release_sweep.py --offline
```

Stage	Checks
`lint` `parse`	Lua 5.0.2 incompatibilities; `luac -p` on every module
`unit`	858 offline tests — decoder, UI, state machine, file paths
`unit-cancel`	the TRIGGER-key cancel latch, one-press recordings, both window sizes, the flow-control loop
`stress`	hostile signals: never silently **wrong**, never **raises**
`tolerance`	recomputes the envelope table printed in the manual
`package` `archive`	rebuilds the `.tspa`, then builds both screens *from the archive* against a mock front end
`manual`	rebuilds every shipped PDF: `docs/MANUAL.pdf`, `docs/REFERENCE.pdf`, `README.pdf`
`hw-matrix`	on the bench, through the app's own Capture button: six frame formats, the standard rate ladder, a 1 kB non-repeating payload, three logic swings, twelve DC offsets
`hw-odd-rates`	nineteen **non-standard** baud rates — 900, 1500, 3600, 8123, 29127, 104857 …
`hw-panel`	every button in every state, with the panel grabbed **before and after each press** and differenced by region — including a one-press recording and a cancel delivered mid-handler by a trigger timer, since the real stop control is a physical key no harness can press
`hw-break`	degenerate signals and contradictory settings — no signal, DC only, all-`0x00`/`0xFF`/`0x55`, a break, 60 mV of swing, 19 Vpp, rates past the ceiling, and six wrong forced settings. A refusal with a reason passes; confident garbage does not

`hw-panel` is the one worth understanding: it checks six things per press — that the handler does not raise, logs no instrument event, returns inside its latency budget, reports what it did, changed the state it was supposed to, *and that the panel actually shows it.* The last is a separate failure from

the one before it, because the UI caches its writes: an app can be right while the operator reads something stale.

Layout

`tsp/`	the app. `serial_core` acquisition, `uart_decode` framing, `chunk_decode` resumable decode, `serial_ui` panel, `serial_app` orchestration
`tools/`	harnesses. `release_sweep.py` is the entry point; the rest are the individual authorities it calls
`docs/`	the manual, instrument references, panel mockups
`notes/`	bring-up log, findings, handoffs

`tsp/midi_decode.tsp` and `tsp/lin_decode.tsp` are complete and tested but **not shipped** in version 1 — the LIN checksum has never been checked against a real frame. Re-adding either is one line in `tools/package_tspa.py`; the app discovers what it has at runtime.

Bench

Three instruments over LAN: the DMM6500 under test, an SDG2122X generating the stimulus, an SDS1204X-E for verifying the stimulus is what it claims to be. `tools/instruments.py` holds the addresses and the hazards worth knowing — repeated large waveform uploads wedge the generator's LAN service, and calibration state is off limits on both.